



Tiếng Việt



Trang chủ Class & Structure Mảng liên tiếp & 2D Array & Queue Đếm đoạn con Tìm kiếm nhị phân & (Sắp xếp Heap)

Đếm đoạn con & cửa sổ với trạng thái

Nội dung

- 1. Đếm đoạn con qua prefix sum + hash map
 - Biến thể quan trọng
- 2. Cửa sổ với trạng thái: số phần tử phân biệt
- 3. Mẹo "đúng K" = " $\leq K$ " trừ " $\leq (K-1)$ "
 - Biến thể tương tự
- 4. "Với mỗi r , đếm số l thỏa"
- 5. Mẫu nhận diện bài
- 6. Bẫy thường gặp
- 7. Tổng kết

Trong khóa Đồng bạn đã học **two pointers / sliding window cơ bản** — tìm đoạn $[l, r]$ tốt nhất thỏa điều kiện. Bài này tập trung vào hai kỹ thuật **mới và mạnh hơn** ở mức Bạc:

1. **Đếm số đoạn con** thỏa một tính chất — thường dùng prefix sum + hash map.
2. **Cửa sổ với trạng thái phức tạp** — duy trì tần suất, số phần tử phân biệt, max/min bên trong cửa sổ.

Kỹ thuật chính: **biến đổi bài toán "với mỗi r , có bao nhiêu l thỏa?"** thành phép tra cứu trong $O(1)$ trung bình.

1. Đếm đoạn con qua prefix sum + hash map

Đề kinh điển: Cho dãy $a_1, \dots, a_n$ (có thể chứa số âm) và K . Đếm số đoạn $[l, r]$ với $\sum_{i=l}^r a_i = K$.

Tại sao two-pointers KHÔNG đủ? Khi mảng có **số âm**, tổng đoạn không đơn điệu theo cửa sổ — không thể "chỉ kéo l tiến lên." Cần kỹ thuật khác.

Ý tưởng: Đặt $P[i] = a_1 + a_2 + \dots + a_i$ (prefix sum, $P[0] = 0$). Khi đó:

$$\sum_{i=l}^r a_i = P[r] - P[l-1]$$



Tiếng Việt



→ Duyệt r từ trái sang phải, dùng `map<long long, int>` đếm tần suất các giá trị $P[j]$ đã thấy.

C++

Python

```
map<long long, long long> cnt;
cnt[0] = 1; // Prefix rỗng tương ứng P[0] = 0
long long P = 0, ans = 0;
for (int r = 1; r <= n; r++) {
    P += a[r];
    ans += cnt[P - K]; // Số j < r với P[j] = P[r] - K
    cnt[P]++;
}
cout << ans;
```

```
from collections import defaultdict
cnt = defaultdict(int)
cnt[0] = 1
P, ans = 0, 0
for x in a:
    P += x
    ans += cnt[P - K]
    cnt[P] += 1
print(ans)
```

Phức tạp: $O(n)$ trung bình.

Biến thể quan trọng

- **Sum chia hết cho M :** $P[r] \equiv P[l-1] \pmod{M}$. Đếm tần suất $P[i] \pmod{M}$.
- **XOR:** P là prefix XOR, đoạn có $\text{XOR} = K \Leftrightarrow P[r] \oplus P[l-1] = K$.

2. Cửa sổ với trạng thái: số phần tử phân biệt

Đề: Tìm đoạn liên tiếp **dài nhất** có **không quá K giá trị phân biệt**.



Tiếng Việt



count > 0 = số phân biệt.

C++

Python

```

map<int, int> freq;
int l = 1, ans = 0;
for (int r = 1; r <= n; r++) {
    freq[a[r]]++; // Thêm vào cửa sổ
    while ((int)freq.size() > K) {
        if (--freq[a[l]] == 0) freq.erase(a[l]);
        l++;
    }
    ans = max(ans, r - l + 1);
}

```

```

from collections import defaultdict
freq = defaultdict(int)
l = 0
ans = 0
for r in range(n):
    freq[a[r]] += 1
    while len(freq) > K:
        freq[a[l]] -= 1
        if freq[a[l]] == 0:
            del freq[a[l]]
        l += 1
    ans = max(ans, r - l + 1)

```

Phức tạp: $O(n \log n)$ với `map`, $O(n)$ trung bình với `unordered_map` / `dict`.

3. Mẹo "đúng K" = " $\leq K$ " trừ " $\leq (K-1)$ "

Đề: Đếm số đoạn có **đúng** K giá trị phân biệt (hoặc đúng K số lẻ, đúng K số chẵn, ...).

Vấn đề: Sliding window tự nhiên đếm **không quá** K , không phải đúng K .



Tiếng Việt



$$\#(\text{đúng } K) = \#(\leq K) - \#(\leq K - 1)$$

Gọi `atMost(K)` = hàm đếm số đoạn có không quá K giá trị phân biệt (dùng sliding window + map). Đáp án = `atMost(K) - atMost(K-1)`.

C++

Python

```
long long atMost(int K) {
    map<int,int> freq;
    int l = 1;
    long long ans = 0;
    for (int r = 1; r <= n; r++) {
        freq[a[r]]++;
        while ((int)freq.size() > K) {
            if (--freq[a[l]] == 0) freq.erase(a[l]);
            l++;
        }
        ans += r - l + 1;
    }
    return ans;
}

long long exactlyK = atMost(K) - atMost(K - 1);
```



Tiếng Việt



```
def at_most(K):
    freq = defaultdict(int)
    l, ans = 0, 0
    for r in range(n):
        freq[a[r]] += 1
        while len(freq) > K:
            freq[a[l]] -= 1
            if freq[a[l]] == 0:
                del freq[a[l]]
            l += 1
        ans += r - l + 1
    return ans

exactly_K = at_most(K) - at_most(K - 1)
```

Đây là mẹo cực kỳ phổ biến — gặp trong hàng trăm bài. Hãy nhớ thật kỹ!

Biến thể tương tự

- Đếm số đoạn có **đúng K số lẻ** $\rightarrow$ `atMostKOdd(K) - atMostKOdd(K-1)` với `atMostKOdd` đếm bằng sliding window đếm số lẻ.
- Đếm số đoạn có tổng $= K$ $\rightarrow$ đã giải bằng prefix + map ở phần 1.

4. "Với mỗi r , đếm số l thỏa"

Khi điều kiện đơn điệu (không có số âm/đặc biệt): dùng sliding window thẳng, đếm $r - l + 1$ ở mỗi bước.

Đề: Đếm số đoạn có tổng $\leq S$ (giả sử $a_i > 0$).

C++

Python



```
int l = 1;
for (int r = 1; r <= n; r++) {
    sum += a[r];
    while (sum > S) sum -= a[l++];
    ans += r - l + 1;
}
```

```
sum_, ans = 0, 0
l = 0
for r in range(n):
    sum_ += a[r]
    while sum_ > S:
        sum_ -= a[l]; l += 1
    ans += r - l + 1
```

Lý do: sau khi l là vị trí nhỏ nhất sao cho $[l..r]$ vẫn thỏa, mọi đoạn $[l..r], [l+1..r], \dots, [r..r]$ đều có tổng $\leq S$ (ngắn hơn $\rightarrow$ tổng nhỏ hơn).

5. Mẫu nhận diện bài

ĐỀ BÀI	KỸ THUẬT
Đếm đoạn có tổng = K (có số âm)	Prefix sum + hashmap
Đếm đoạn có tổng chia hết cho M	Prefix mod M + hashmap
Đếm đoạn có XOR = K	Prefix XOR + hashmap
Đoạn dài nhất có $\leq K$ phân biệt	Sliding + map đếm tần suất
Đếm đoạn có đúng K phân biệt	<code>atMost(K) - atMost(K-1)</code>
Đếm đoạn tổng $\leq S$ ($a_i > 0$)	Sliding, $\sum (r - l + 1)$
Max/min trong mọi cửa sổ độ dài k	Monotonic deque (đã học bài 4)



Tiếng Việt



- **Quên** `cnt[0] = 1` trong prefix-sum hashmap → bỏ sót đoạn bắt đầu từ vị trí 1.
- **Trần long long**: tổng prefix có thể lớn — luôn dùng `long long`.
- **Hash collision attack**: `unordered_map` có thể bị TLE — đổi sang `map` nếu n vừa phải hoặc dùng custom hash.
- **At-most-K**: nhớ trừ `atMost(K-1)`, không phải `atMost(K) - 1`.
- **Sliding cần đơn điệu**: nếu mảng có số âm và bài hỏi tổng, two-pointers thông thường KHÔNG dùng được — phải prefix + map.

7. Tổng kết

Bài này nâng cấp two-pointers / sliding window từ "tìm cửa sổ tốt nhất" sang **đếm tất cả các cửa sổ** và **duy trì trạng thái phức tạp trong cửa sổ**. Hai chiến thuật chính:

- **Prefix sum + hashmap**: cho bài toán đếm tổng/XOR/mod (xử lý được số âm).
- **Sliding window + map/multiset**: cho bài toán có "trạng thái" trong cửa sổ (số phân biệt, tần suất).
- **At-most-K trick**: chuyển "đúng K" về hai lần " $\leq K$ ".

Đây là những kỹ thuật bạn sẽ gặp **rất nhiều** ở các bài tầm trung trở lên!

Bài tập

	Đếm Kí Tự	0 / 10
	Đoạn con (HSG THPT Hà Tĩnh 2023)	0 / 15
	Chùm đèn (HSG 9 Đà Nẵng 2023-2024)	0 / 20
	Đoạn đường nhàm chán	0 / 25
	CSES - Subarray Sums II Tổng đoạn con II	0 / 25
	CSES - Subarray Divisibility Tính chia hết của đoạn con	0 / 25
	CSES - Subarray Distinct Values Giá trị phân biệt trong đoạn con	0 / 30
	Dãy dài nhất	0 / 30